



UNIVERSITÉ DE
MONTPELLIER



STATISTIQUE
SCIENCE DES DONNÉES BIOSTATS
UNIVERSITÉ DE MONTPELLIER



FACULTÉ DES SCIENCES
DE MONTPELLIER

M2 STATISTIQUES ET SCIENCES DES DONNÉES – BIOSTATISTIQUES –

RAPPORT DE STAGE

Titre du stage

Guillaume BOULAND

Tuteur académique :

Nicolas Meyer

Tuteurs professionnels :

Mai Nguyen-Chi

Benjamin Charlier

Jury :

1

2

3



2023 – 2024

Remerciements

blabla

Résumé

Résumé

Abstract

Table des matières

1	Introduction	5
	Cadre du stage	5
	Contexte	5
	Question	5
	Objectifs	5
2	Mactrack2	6
2.1	Introduction	6
2.2	Éléments théoriques	7
	Image numérique	7
	Différentes approches de segmentation d'images	7
	Cartesian Genetic Programming (CGP)	9
	Kartezio	9
3	SAM-2	10
3.1	Introduction	10
3.2	Fine-tuning	11
	3.2.1 Définition	11
	3.2.2 Jeu de données	11
	3.2.3 Hyper-paramètres	12
	3.2.4 Fonctions de perte	12
	3.2.5 Optimisation	15
	3.2.6 Résultats et comparaisons	16
3.3	Propagation dans une vidéo	16
4	Conclusion	17
5	Matériel et Méthodes	18

Chapitre 1

Introduction

Cadre du stage

Contexte

mehari, nguyen ...
lignée transgénique obtention des images, microscopie

Question

Objectifs

Chapitre 2

Mactrack2

2.1 Introduction

Pour répondre à la question biologique posée, nous devons développer un outil qui permettrait de segmenter des macrophages dans des vidéos. Pour ce faire, nous avons tout d'abord réfléchi au problème de manière séquentielle, c'est-à-dire image par image. En poursuivant le travail déjà effectué au laboratoire par l'ancien stagiaire Axel de Montgolfier[1], étudiant en Master 2 Statistiques et Sciences des Données qui était présent durant l'année 2023-2024, nous avons amélioré le programme MacTrack créé par ses soins, en le rendant plus robuste avec notamment un meilleur jeu de données pour l'entraînement des modèles. Nous pouvons maintenant visualiser la qualité des modèles entraînés. Des scripts plus intuitifs pour les débutants, qui sont la cible de ce projet, ont été ajoutés ainsi qu'une documentation complète sur un site internet, disponible sur le dépôt GitHub du projet ([2]). Un post-processing a été implémenté pour corriger les erreurs de segmentation des macrophages. Ces erreurs que nous avons rencontrées étaient souvent dues à des problèmes au niveau de l'image de microscopie, comme par exemple la présence d'un fond bruité. Ce process sert aussi pour obtenir une segmentation continue dans le temps, sans changement de numérotation, afin d'obtenir la meilleure analyse possible des flux calciques présent dans ces macrophages.

Ainsi, toutes ces améliorations constituent **MacTrack2**, une librairie développée en Python, téléchargeable et utilisable simplement via les scripts et le jeu de données que nous fournissons en exemples. Elle se base principalement sur la librairie *Kartezio*[3], qui a été créée pour proposer une nouvelle approche de la segmentation d'objets dans des images microscopiques en se basant sur les principes du *Cartesian Genetic Programming* (CGP)[4]. À l'aide de cette méthode, *Kartezio* est capable de générer des pipelines de segmentations qui sont à la fois peu gourmands en ressources, notamment en quantité d'images pour l'entraînement ou en temps computationnel, interprétables puisque cette librairie utilise une liste de fonction issues d'*OpenCV*[5, 6], une bibliothèque libre de fonctions spécialisées dans le traitement d'images, et performants.

2.2 Éléments théoriques

Image numérique

Les images que nous obtenons par microscopie confocale à l'aide de la lignée transgénique, comme vu dans l'**Introduction**, sont des images en 2 dimensions (2D) où l'on retrouve deux canaux de couleurs : le rouge pour la présence du macrophage (en bordure des cellules) et le vert pour la présence de calcium dans le cytoplasme du macrophage. Cependant, ces couleurs ont été ajoutées artificiellement par soucis de visualisation à l'aide du logiciel *ImageJ*[7] et plus particulièrement *Fiji*[8], qui ajoute un certain nombre d'options à ce logiciel, sous forme de plugins et macros. Ces couleurs ont été spécifiquement choisies car les protéines fluorescentes (*mCherry* pour le rouge et *GFP* (*Green Fluorescent Protein*) pour le vert) utilisées dans la lignée transgénique sont activées par la stimulation provoquée par un laser à une certaine longueur d'onde qui correspond à ces couleurs.

Les images que nous recevons, après avoir séparé les canaux, sont donc des images en niveaux de gris, où chaque pixel de celle-ci est un nombre réel entre 0 et 255, avec 0 représentant le noir et 255 le blanc. Cela nous évite la complexité de traiter une image RGB (Red, Green, Blue) où chaque pixel est un triplet d'intensité de chacune des trois couleurs (Fig. 2.1).

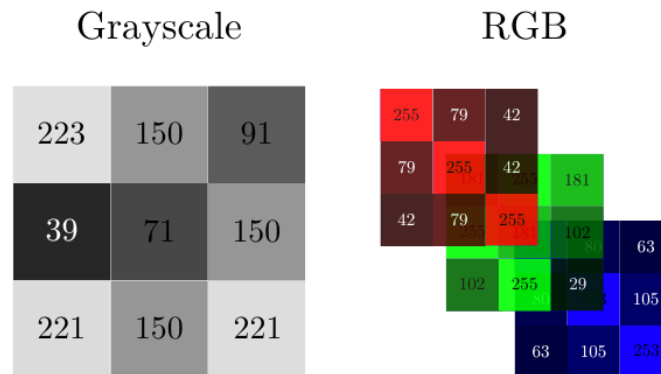


Figure 2.1. Représentations matricielles d'une image en niveaux de gris (*Grayscale*) et d'une image RGB. Figure issue de [9]

Différentes approches de segmentation d'images

La segmentation d'images est un problème de vision par ordinateur (*Computer Vision*) qui consiste à diviser une image en plusieurs régions ou objets d'intérêt. Ici, les objets d'intérêt sont des macrophages, mais la segmentation est un problème plus général, applicable à de nombreux domaines. Les régions ou objets d'intérêts peuvent prendre plusieurs formes comme par exemple des rectangles permettant de délimiter la présence d'un objet. Elle sert à **détecter** tous les objets d'intérêt dans une image et peut même être utilisée pour labelliser ces objets [10, 11, 12, 13].

La **segmentation sémantique** est une autre approche de segmentation d'images où on affecte à chaque pixel une classe d'objet, et ce pour chaque objet de nature

(ou sémantique) différente. Elle ne fait donc pas la différence entre deux voitures de couleur ou modèle différents par exemple, mais les classifera chacun comme étant une voiture.

La **segmentation d'instance** est une approche plus complexe qui se concentre sur la distinction des instances, c'est-à-dire des objets individuels, même les instances occluses de la même classe d'objet. Si on reprend notre exemple, les voitures sont maintenant segmentées individuellement, même si elles sont identiques, puisqu'elles sont vues comme des objets différents mais appartenant à la même classe « voiture ».

La **segmentation panoptique** combine les deux précédentes. Elle englobe à la fois la classification sémantique de chacun des pixels de l'image et la délimitation de chaque instance d'objet, mais son coût de calcul est plus élevé.

Dans notre cas, nous cherchons à segmenter des macrophages, donc sur le canal rouge de l'image, pour ensuite analyser les flux calciques de ceux-ci à partir de la segmentation obtenue. Nous avons donc besoin d'une segmentation d'instances car nous voulons détecter chaque macrophage tout en faisant attention à ne pas les confondre entre eux. De plus, nous avons une difficulté en plus car jusqu'à présent, nous avons seulement traité de segmentation d'images et non de vidéos. Or, les macrophages étant des cellules immunitaires très mobiles, ils pourraient se regrouper ou se séparer à tout moment de la vidéo. Donc la qualité de la segmentation est primordiale pour pouvoir au mieux retranscrire les comportements réels des macrophages.

Cependant, la segmentation d'instances fait appel à des modèles, comme les réseaux de neurones convolutifs par exemple, qui demandent un grand nombre d'images annotées. Or, annoter à la main des milliers d'images demande un temps énorme que nous n'avons pas. De plus, pour pouvoir segmenter autant d'images, il faut au préalable réaliser toutes les manipulations telles que la reproduction de poissons, le triage des œufs, la sélection des larves, la coupe de la nageoire caudale en fonction de la condition que l'on cherche à observer, l'observation et l'enregistrement microscopique pendant 40 minutes (durée des films que nous avons) à 3, 6, 12, 24, 48 et 72 heures post-amputation, pour chacun des poissons sélectionnés et pour finir le traitement des films sortis de microscopie pour qu'ils soient exécutables. Cela demande un coût humain et financier énorme que nous n'avons pas. En se tournant vers les modèles de classification sémantique, il avait été découvert l'année précédente *Kartezio*[3], une librairie Python, qui a pour promesse de segmenter, et ce en concurrençant les modèles basés sur des réseaux de neurones bien connus du domaine comme *Cellpose*[14, 15, 16], *Mask R-CNN*[17] ou encore *Stardist*[18], des objets dans des images biomédicales. Plus précisément, *Kartezio* a pour avantage de générer des pipelines de segmentation robustes et interprétables à partir de peu d'images annotées, en se basant sur le CGP.

Cartesian Genetic Programming (CGP)

Kartezio

nous on veut instance mais trop cher donc on ruse sémantique into instance
grace a kartezio qui utilise cgp subsec cgp

Chapitre 3

SAM-2

3.1 Introduction

SAM-2 (*Segment Anything Model 2*) est un modèle de segmentation d'images et de vidéos développé par Meta AI. Il s'agit d'une version améliorée de la première version du modèle, SAM[19], généralisant le problème de segmentation d'images au domaine des vidéos (*VOS : Video Object Segmentation*). Entraîné sur plus de 50 000 vidéos pour 600 000 masques (annotations), SAM-2 est capable de segmenter des objets dans des vidéos de manière précise et efficace. L'architecture du modèle (voir Fig. 3.1) est une architecture basée sur les Transformers[20]. Elle comprend un *image encoder* qui est un Vision Transformer hiérarchique appelé Hiera[21], un *prompt encoder* qui prend en compte des clics (positifs ou négatifs), des boîtes ou des masques permettant de définir un objet d'intérêt dans une image et un *mask decoder* qui prend ces entrées et en ressort un masque de segmentation de l'objet sélectionné. Il utilise une mémoire en flux (*streaming memory*). Celle-ci permet de traiter les images d'une vidéo en temps réel, une par une. Ainsi, il peut garder en mémoire, à l'aide de la *memory attention* et de la *memory bank*, les objets et leurs interactions au cours du temps, permettant ainsi de générer des masques plus précis et de les corriger en fonction des images précédentes. La *memory attention* permet de « préparer » l'image en prenant en compte la *memory bank*, qui garde en mémoire les précédentes images, mais aussi les nouveaux points, masques, boîte, que l'utilisateur peut ajouter à chaque image.

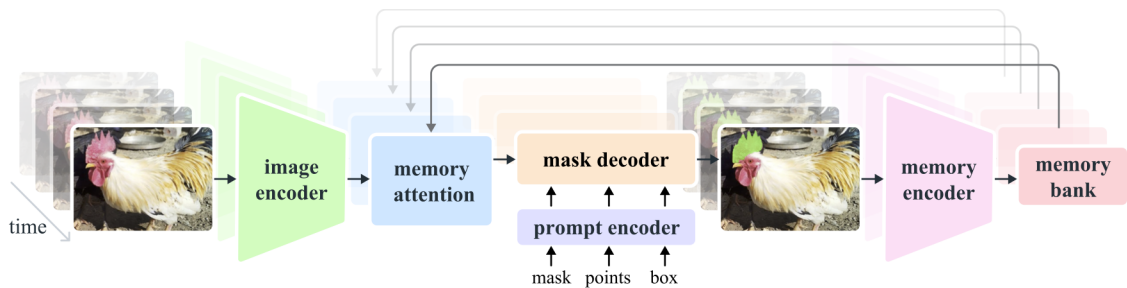


Figure 3.1. Architecture de SAM-2 (figure issue de [22]).

Ainsi, SAM-2 est considéré (au même titre que SAM) comme un modèle de fondation (*foundation model*)[23], c'est-à-dire un modèle pré-entraîné sur une grande

quantité de données, capable de généraliser à de nombreuses tâches différentes. Il est conçu pour être utilisé comme une base pour des tâches de segmentation en gardant de bonnes performances pour l'ensemble des applications de segmentation d'images et de vidéos.

Un des problèmes que nous rencontrons avec SAM-2 est que l'utilisateur doit, à la main, sélectionner des objets d'intérêt dans une ou plusieurs images et si durant la vidéo un nouvel objet apparaît, il ne sera pas détecté tant que l'utilisateur ne l'a pas indiqué. Ainsi, nous allons utiliser la génération automatique de masques mise à disposition pour détecter les macrophages de manière automatique sur des images.

performances nulles donc fine tuning

3.2 Fine-tuning

3.2.1 Définition

Le *fine-tuning* (ou *affinage* en français) est une technique d'apprentissage supervisé qui consiste à modifier les poids d'un modèle pré-entraîné sur une tâche spécifique. Elle est souvent utilisée pour adapter un modèle généraliste à une tâche particulière. Ici, SAM-2 a été entraîné sur des photos qui n'ont rien à voir avec la problématique biologique que nous rencontrons. Par conséquent, afin d'obtenir une bonne segmentation des macrophages, il est nécessaire de modifier les poids du modèle pour obtenir une version spécialisée dans la détection de macrophages. Cela nous évite de devoir entraîner notre propre modèle en partant de rien, ce qui nous demanderait beaucoup trop de données que nous n'avons pas. Mais le nombre de données nécessaires pour obtenir un bon affinage n'est pas négligeable.

Le fine-tuning de SAM-2 agit seulement sur les poids du *prompt encoder* et du *mask encoder*, c'est-à-dire les parties du modèle qui prennent en entrée les points, masques, boîtes, et qui génèrent les masques de segmentation. Nous spécifions donc la prise en charge des entrées données par l'utilisateur, mais pas la partie qui encode l'image ni la partie qui mémorise les images précédente une fois le fine-tuning fini et que l'on cherche à segmenter une vidéo entière.

3.2.2 Jeu de données

Les annotations sont sous la forme d'images où chaque macrophage est détourné à la main sur le logiciel *ImageJ* (et plus particulièrement *Fiji*[8]). Les détournages sont ensuite séparés par label, afin de s'assurer qu'un macrophage ne soit pas confondu avec un autre (Fig. 3.2).

Malgré que le fine-tuning de SAM-2 ne nécessite pas autant de données que l'entraînement d'un modèle de segmentation complet, il est quand même nécessaire de disposer d'un nombre conséquent de données. Ici, nous avons besoin d'images et de leur segmentation manuelle. Or, le temps imparti pour ce stage ne nous permet pas de composer un jeu de données complet qui satisferait ces besoins. Par conséquent,

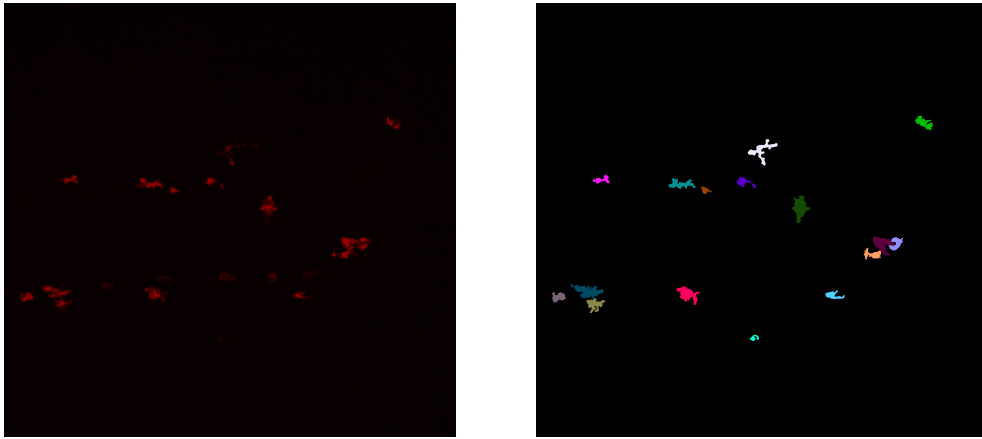


Figure 3.2. Image d'origine (à gauche) comprenant des macrophages et leur détourage (à droite).

nous avons utilisé le programme MacTrack2 qui nous a permis de segmenter une vidéo de macrophages. Ainsi, nous disposons de 297 images et masques de macrophages (dont 266 faisant partie d'une seule vidéo) pour ré-entraîner les poids du modèle.

3.2.3 Hyper-paramètres

Les hyper-paramètres, dans un modèle, constituent les paramètres non appris par le modèle durant l'entraînement (ou le ré-entraînement). Ils sont fixés à l'avance et influencent son comportement et par extension ses performances. Les hyper-paramètres du pré-entraînement et de l'entraînement complet de SAM-2 sont disponibles dans leur papier[22] page 19.

Pour le fine-tuning, les sous-modules affectés sont le *mask decoder* et le *prompt encoder*. Nous utilisons l'optimiseur AdamW[24] avec un taux d'apprentissage (*learning rate*) de 0.0001 et une dégradation des poids (*weight decay*) de 0.00001, utile pour la régularisation L2. Le nombre total d'itérations est de 20 000, avec un nombre de pas pour l'accumulation des gradients (*accumulation step*) avant leur mise à jour égal à 4 steps. Le *scheduler* permet d'ajuster le *learning rate* durant l'entraînement pour améliorer la convergence du modèle. Ici nous utilisons un *stepLR scheduler*, ce qui signifie que tous les k steps, le *learning rate* est multiplié par un facteur γ . Nous avons $k = 500$ et $\gamma = 0.1$. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas.

Contrairement à ce que l'on peut retrouver habituellement dans l'entraînement de modèles, la taille du batch est égale à une seule image. En effet, à chaque pas ...

3.2.4 Fonctions de perte

Pour entraîner un modèle, il est nécessaire de définir une fonction de perte $\mathcal{L}$ à minimiser. Elle est utilisée pour quantifier la qualité des prédictions du modèle par rapport à la « vérité » (ground truth). Dans le cas du fine-tuning de SAM-2, nous

utilisons une combinaison de deux fonctions de perte : la *binary cross-entropy loss* qui sert de perte pour la segmentation et la *score loss* qui est utilisée en complément pour évaluer la qualité des prédictions des masques à partir de la métrique IoU (*Intersection over Union*). L'IoU est une métrique grandement utilisée dans les problèmes de segmentation d'images. Elle est définie, pour un macrophage k dans une image, de la manière suivante et peut être illustrée par la Fig. 3.3 :

$$\text{IoU}(M_k, G_k) = \frac{|M_k \cap G_k|}{|M_k \cup G_k|} = \frac{\sum_{i,j} M_k[i, j] \cdot G_k[i, j]}{\sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]}$$

avec :

- $i \in \{1, \dots, H\}$ et $j \in \{1, \dots, W\}$ la hauteur et la largeur respectivement de l'image,
- $M_k = (M_k[i, j])_{i,j}$ le masque prédit pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel vaut 1 s'il est dans le masque prédit, c'est-à-dire que si la probabilité que le pixel (i, j) appartienne au masque de l'objet k est supérieure à 0.5, sinon il vaut 0. On a donc :

$$M_k[i, j] = \begin{cases} 1 & \text{si } p > 0.5, \\ 0 & \text{sinon.} \end{cases}$$

avec p la probabilité que le pixel (i, j) appartienne au masque de l'objet k ,

- $G_k = (G_k[i, j])_{i,j}$ le masque de vérité pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel (élément de la matrice) vaut 1 s'il est dans le masque de vérité, sinon il vaut 0,
- $|M_k \cap G_k| = \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'intersection entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels en commun entre les deux masques,
- $|M_k \cup G_k| = \sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'union entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels dans au moins un des deux masques,
- $\sum_{i,j} M_k[i, j]$ le nombre de pixels du masque prédit,
- $\sum_{i,j} G_k[i, j]$ le nombre de pixels du masque de vérité.

En moyennant sur tous les objets k d'une image, on obtient la mIoU (IoU moyenne), métrique très utilisée, pour cette image :

$$\text{mIoU} = \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k)$$

Ainsi, nous pouvons définir la *score loss* comme suit :

$$\mathcal{L}_{\text{score}} = \frac{1}{K} \sum_{k=1}^K |s_k - \text{IoU}(M_k, G_k)|$$

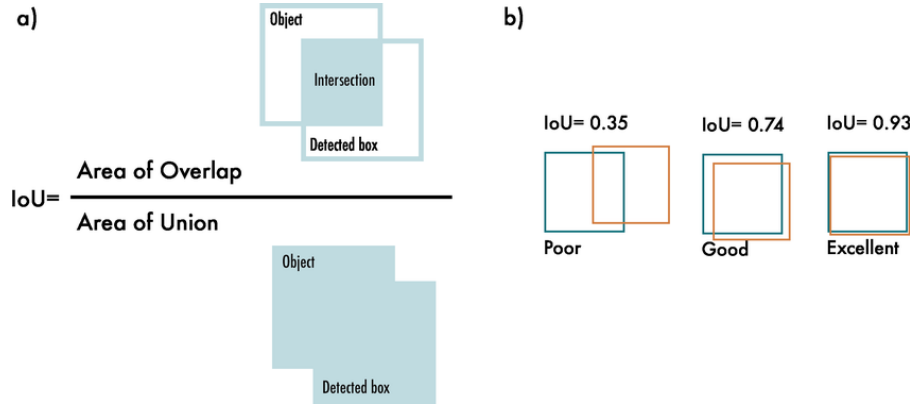


Figure 3.3. Schématisation de l'IoU (figure issue de [25]).

avec K le nombre total de masques (donc de macrophages ou d'objets) dans l'image et s_k le score prédit pour l'objet k à partir de M_k .

Nous comparons donc le score prédit par le modèle pour chaque masque à la véritable qualité de chacun d'eux (l'IoU). Puisque'on cherche à minimiser cette perte, on aimerait que pour tout k , $s_k \simeq \text{IoU}(M_k, G_k)$ pour que la perte soit proche de 0.

Si $K = 0$, alors cela signifie qu'il n'y a pas de masques dans l'image, donc pas de macrophages. Un tel cas peut être rencontré, mais nous avons décidé de ne pas l'inclure dans le fine-tuning, car il n'apporte pas d'information utile concernant la segmentation des macrophages.

D'un autre côté, la *binary cross-entropy loss* $\mathcal{L}_{BCE}$ est une fonction de perte généralement utilisée dans des problèmes de classification binaire, et notamment dans les problèmes de segmentation d'images. Elle est utilisée pour quantifier la différence entre les prédictions du modèle et les véritables annotations (masques de vérité). Elle est définie comme suit :

$$\mathcal{L}_{BCE} = -\frac{1}{K \times H \times W} \sum_{k=1}^K \sum_{i=1}^H \sum_{j=1}^W y_{k,i,j} \log \hat{y}_{k,i,j} + (1 - y_{k,i,j}) \log(1 - \hat{y}_{k,i,j})$$

avec :

- $y_{k,i,j} \in \{0, 1\}$ la valeur du pixel (i, j) du masque de vérité pour l'objet k . On peut notamment remarquer que la matrice $G_k = (y_{k,i,j})_{i,j}$ est une matrice binaire (composée de 0 et de 1) de même dimension que les dimensions de l'image ($H \times W$).
- $\hat{y}_{k,i,j} \in [0, 1]$ est la probabilité que le pixel (i, j) appartienne au masque de l'objet k . Elle est obtenue par la transformation sigmoïde appliquée à la prédiction des masques du modèle, qui sont des logits générés par le *mask decoder* (voir Tab. 3.1). On a donc :

$$\hat{y}_{k,i,j} = \sigma(z_{k,i,j}) = \frac{1}{1 + \exp(-z_{k,i,j})}$$

avec $z_{k,i,j}$ le logit du pixel (i, j) du masque prédit pour l'objet k .

Logit ($z_{k,i,j}$)	Interprétation	Sigmoid ($\hat{y}_{k,i,j}$)
+10	Très sûr qu'il s'agit d'un objet	≈ 1
+1	Assez confiant qu'il s'agit d'un objet	≈ 0.75
0	Incertitude totale	0.5
-1	Assez confiant qu'il s'agit du fond de l'image	≈ 0.25
-10	Très sûr qu'il s'agit du fond de l'image	≈ 0

Table 3.1. Différents exemples des valeurs que peuvent prendre les sorties du *mask decoder* (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde).

- H et W la hauteur et la largeur de l'image.
- K le nombre total de masques (donc de macrophages ou d'objets) dans l'image.

Interprétations fonction de perte ...ressemble a kullbackleibler mais différences ...

Finalement, la fonction de perte totale est une combinaison de ces deux dernières :

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda \mathcal{L}_{\text{score}}$$

avec λ un hyper-paramètre qui permet de pondérer l'importance de la *score loss* par rapport à la *binary cross-entropy loss*. Dans notre cas, nous avons fixé λ à 0.05.

3.2.5 Optimisation

Une fois la perte calculée, elle est divisée par le nombre de pas d'accumulation des gradients, soit 4 dans notre cas, comme défini dans la partie 3.2.3 introduisant les hyper-paramètres. Cela nous permet d'éviter de multiplier le gradient total par ce facteur. C'est utilisé lorsque l'on accumule des gradients sur plusieurs images (ici 4 fois une image) pour que la somme finale corresponde en réalité à une moyenne sur ce batch étendu. Ainsi, nous simulons un batch de taille 4 alors qu'en réalité nous n'avons qu'une seule image par step.

Ensuite, les gradients sont calculés en utilisant la méthode *backward* du package PyTorch[26], à partir de la perte totale $\mathcal{L}$ divisée par cet *accumulation step*. Puis, tous ces 4 steps, nous mettons à jour l'optimiseur AdamW. Le nombre de pas avant la mise à jour du *learning rate* est de 500. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas. Ensuite, les gradients sont remis à zéro pour le prochain pas d'accumulation.

Pour finir, nous calculons l'IoU moyenne au pas ℓ comme étant :

$$\text{mIoU}_\ell = \begin{cases} 0.9 \cdot \text{mIoU}_{\ell-1} + 0.1 \cdot \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) & \text{si } k > 0, \\ 0 & \text{sinon.} \end{cases}$$

3.2.6 Résultats et comparaisons

graphes miou et loss avant division par accumulation step

3.3 Propagation dans une vidéo

Chapitre 4

Conclusion

Disponibilité du code

Le code de MacTrack2 est disponible, ainsi que deux scripts détaillés pour la création de modèle et l'analyse d'une vidéo, avec un jeu de données d'exemple pour chacun des deux scripts, à l'adresse suivante : <https://github.com/guibouland/MacTrack2>.

Un site internet est disponible sur ce dépôt GitHub, regroupant toute la documentation et des instructions détaillées pour les utilisateurs plus novices, notamment les biologistes traitant la même problématique, qui sont la cible principale de ce projet. Il servira aussi aux futurs membres du laboratoire LPHI qui souhaitent utiliser MacTrack2 pour leurs propres projets.

Chapitre 5

Matériel et Méthodes

tests tests[3, 19, 22, 24]

Acquisition des images

microscope . . .norma merci

Fine-tuning de SAM-2

L'optimiseur AdamW a été utilisé avec comme hyper-paramètres un taux d'apprentissage (*learning rate*) de 0.0001, une dégradation de pondération (*weight decay*) de 0.0001. L'entraînement a été effectué avec 20000 itérations. Nous avons utilisé un GPU Nvidia A10 Tensor Core avec 24 Go de mémoire.

Bibliographie

- [1] Axel de MONTGOLFIER. *Github - Stage LPHI 2024*. URL : <https://github.com/Axeldmont/Stage-LPHI-2024>.
- [2] Guillaume BOULAND. *Github - MacTrack2*. URL : <https://github.com/guibouland/MacTrack2>.
- [3] Kévin CORTACERO et al. « Evolutionary design of explainable algorithms for biomedical image segmentation ». In : *Nature Communications* 14.1 (2023), p. 7112.
- [4] Julian MILLER et Andrew TURNER. « Cartesian Genetic Programming ». In : *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. GECCO Companion '15. Madrid, Spain : Association for Computing Machinery, 2015, p. 179-198. ISBN : 9781450334884. DOI : [10.1145/2739482.2756571](https://doi.org/10.1145/2739482.2756571).
- [5] OpenCV TEAM. *OpenCV : Open source Computer Vision Library*. <https://opencv.org/>.
- [6] OpenCV TEAM. *OpenCV : Open source Computer Vision Library - Python*. <https://github.com/opencv/opencv-python>.
- [7] Caroline A SCHNEIDER, Wayne S RASBAND et Kevin W ELICEIRI. « NIH Image to ImageJ : 25 years of image analysis ». In : *Nature methods* 9.7 (2012), p. 671-675.
- [8] Johannes SCHINDELIN et al. « Fiji : an open-source platform for biological-image analysis ». In : *Nature Methods* 9.7 (2012), p. 676-682. DOI : [10.1038/nmeth.2019](https://doi.org/10.1038/nmeth.2019).
- [9] NAMITA. *Convolutional Neural Networks*. Medium blog post. Déc. 2023. URL : <https://medium.com/@namitabagri/convolutional-neural-networks-fb0abfdc1c7c>.
- [10] Qing JIANG et al. *T-Rex2 : Towards Generic Object Detection via Text-Visual Prompt Synergy*. 2024. arXiv : [2403.14610](https://arxiv.org/abs/2403.14610) [cs.CV]. URL : <https://arxiv.org/abs/2403.14610>.
- [11] Tianhe REN et al. *Grounded SAM : Assembling Open-World Models for Diverse Visual Tasks*. 2024. arXiv : [2401.14159](https://arxiv.org/abs/2401.14159) [cs.CV].
- [12] Shilong LIU et al. « Grounding dino : Marrying dino with grounded pre-training for open-set object detection ». In : *arXiv preprint arXiv :2303.05499* (2023).
- [13] Tianhe REN et al. *Grounding DINO 1.5 : Advance the "Edge" of Open-Set Object Detection*. 2024. arXiv : [2405.10300](https://arxiv.org/abs/2405.10300) [cs.CV].
- [14] Carsen STRINGER et al. « Cellpose : a generalist algorithm for cellular segmentation ». In : *Nature methods* 18.1 (2021), p. 100-106.
- [15] Marius PACHITARIU et Carsen STRINGER. « Cellpose 2.0 : how to train your own model ». In : *Nature methods* 19.12 (2022), p. 1634-1641.

- [16] Carsen STRINGER et Marius PACHITARIU. « Cellpose3 : one-click image restoration for improved cellular segmentation ». In : *Nature Methods* (2025), p. 1-8.
- [17] Kaiming HE et al. « Mask r-cnn ». In : *Proceedings of the IEEE international conference on computer vision*. 2017, p. 2961-2969.
- [18] Uwe SCHMIDT et al. « Cell detection with star-convex polygons ». In : *Medical image computing and computer assisted intervention–MICCAI 2018 : 21st international conference, Granada, Spain, September 16-20, 2018, proceedings, part II 11*. Springer. 2018, p. 265-273.
- [19] Alexander KIRILLOV et al. « Segment Anything ». In : (2023). arXiv : [2304.02643](https://arxiv.org/abs/2304.02643) [cs.CV]. URL : <https://arxiv.org/abs/2304.02643>.
- [20] Ashish VASWANI et al. *Attention Is All You Need*. 2023. arXiv : [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL : <https://arxiv.org/abs/1706.03762>.
- [21] Chaitanya RYALI et al. *Hiera : A Hierarchical Vision Transformer without the Bells-and-Whistles*. 2023. arXiv : [2306.00989](https://arxiv.org/abs/2306.00989) [cs.CV]. URL : <https://arxiv.org/abs/2306.00989>.
- [22] Nikhila RAVI et al. « SAM 2 : Segment Anything in Images and Videos ». In : *arXiv preprint arXiv :2408.00714* (2024). URL : <https://arxiv.org/abs/2408.00714>.
- [23] Rishi BOMMASANI et al. *On the Opportunities and Risks of Foundation Models*. 2022. arXiv : [2108.07258](https://arxiv.org/abs/2108.07258) [cs.LG]. URL : <https://arxiv.org/abs/2108.07258>.
- [24] Ilya LOSHCHILOV et Frank HUTTER. « Decoupled Weight Decay Regularization ». In : (2019). arXiv : [1711.05101](https://arxiv.org/abs/1711.05101) [cs.LG]. URL : <https://arxiv.org/abs/1711.05101>.
- [25] Juan TERVEN et Diana-Margarita CORDOVA-ESPARZA. *A Comprehensive Review of YOLO : From YOLOv1 to YOLOv8 and Beyond*. Avr. 2023. DOI : [10.48550/arXiv.2304.00501](https://doi.org/10.48550/arXiv.2304.00501).
- [26] Adam PASZKE et al. « PyTorch : An Imperative Style, High-Performance Deep Learning Library ». In : (déc. 2019). DOI : [10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).